

weak_ptr — Խորը ուղեցույց

C++ Smart Pointer presentation

Այս ուղեցույցը մանրամասն բացատրում է `std::weak_ptr`-y՝ ինչ է, ինչու է պետք, circular reference-ի լուծում, `lock()`, `expired()`, օրինակներով և interview հարցերով.

1. Ի՞նչ է weak_ptr-ը

`std::weak_ptr`-y "tulacrak" (non-owning) smart pointer է, ընդհանուր առմամբ `shared_ptr`-ի **karavarac resource-in**, ինչպես **CHI achecnum reference count-y**.

```
#include <memory>

auto sp = std::make_shared<int>(10);    // strong count = 1
std::weak_ptr<int> wp = sp;              // strong count DEՐ 1 (weak chi achecnum)

cout << sp.use_count();                 // 1 (weak_ptr chi hashvvum)
```

`weak_ptr`-y "ditord" է՝ գտնվող resource-ի մասին, ինչպես ownership չունի, ինչպես `shared_ptr`-ի կարգով.

2. Ինչու է պետք — Circular Reference-ի լուծում

`shared_ptr`-ի `amenamenc problem-y` circular reference (leak)`. `weak_ptr`-y լուծում է այն.

Problem (shared_ptr-ov)

```
struct Node {
    std::shared_ptr<Node> next;
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b count = 2
b->next = a;    // a count = 2
// scope verj -> count 1 (irar pahum) -> ERBEK 0 -> LEAK
```

Lucumy (weak_ptr-ov)

```
struct Node {
    std::weak_ptr<Node> next;    // weak -> count chi achecnum
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b strong count DER 1
b->next = a;    // a strong count DER 1
// scope verj -> count 0 -> DELETE -> CHKA LEAK
```

Parent-child kaperum. parent -> child `shared_ptr`, child -> parent `weak_ptr` (kotrum e cycle-y).

3. lock() — anvtang mutq resource-in

`weak_ptr`-ic unix ches karo dereference anel (resource-y karo e jnjvac linel). Petq e `lock()` -> veradarcnum `shared_ptr` (ete resource-y kar e).

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;

if (auto locked = wp.lock()) {    // shared_ptr veradarcnum (count++ jamanakavor)
    cout << *locked;              // anvtang -- resource kar e
} else {
    cout << "resource jnjvac e";
}
```

`lock()` -y. ete resource-y ka' e -> valid shared_ptr; ete jn'vac -> nullptr shared_ptr.

4. expired() — stugum resource-y ka' e te che

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;

cout << wp.expired();    // false (ka' e)
sp.reset();              // strong count 0 -> resource DELETE
cout << wp.expired();    // true (jn'vac e)
```

`expired()` -> `true` ete resource-y jn'vac e (strong count == 0).

5. Karevor metodner

```
std::weak_ptr<int> wp = sp;

wp.lock();              // shared_ptr veradarcnum (kam nullptr)
wp.expired();           // resource jn'vac e?
wp.use_count();         // qani STRONG owner ka
wp.reset();             // weak-y datarkacnum
```

weak_ptr-ic **CHES** karox unix `*wp` kam `wp->` -- petq e `lock()`.

6. Real orinak — Observer / Cache

```
class Cache {
    std::unordered_map<int, std::weak_ptr<Data>> cache;
public:
    std::shared_ptr<Data> get(int key) {
        if (auto sp = cache[key].lock())    // deŕ kaŕ e?
            return sp;
        auto sp = loadFromDisk(key);        // veranorogel
        cache[key] = sp;                    // weak -> chi erkaracnum kyanqy
        return sp;
    }
};
```

Cache-y weak_ptr e pahum -> chi xangarum resource-i azatmany, bayc karoŕ e "veragtnel" ete deŕ kaŕ e.

7. Interview harcer & patasxanner

weak_ptr inch e?

Non-owning smart pointer vor cuyc e talis shared_ptr-i resource-in bayc chi achecnum strong count-y; chi erkaracnum kyanqy.

Inchu e petq?

Circular reference-i lucum (leak); observer/cache patterns.

Circular reference inchpes e lucum?

Mek uxxutyamb weak_ptr (parent->child shared, child->parent weak) -> cycle kotrvum -> count 0 -> delete.

lock() inch e anum?

Veradarcnum shared_ptr (count++), ete resource kar' e; haka'ak depqum nullptr. Anvtang mutqi dzev.

Inchu unix ches dereference anum weak_ptr?

Resource-y karox e jn'vac linel; petq e lock() -> stugel kar' e te che.

expired() inch e?

true ete resource jn'vac e (strong count 0).

weak_ptr achecnum e use_count-y?

Voch -- miayn strong (shared_ptr) count. weak count a'ancin e.

Cheat Sheet

`weak_ptr = NON-OWNING (chi achecnum strong count, chi erkaracnum kyanq)`

<code>wp = shared_ptr</code>	-> weak reference (strong count DER nuyny)
<code>wp.lock()</code>	-> shared_ptr (kam nullptr ete jn'vac) -- ANVTANG mutq
<code>wp.expired()</code>	-> resource jn'vac e? (strong count 0)
<code>wp.use_count()</code>	-> qani STRONG owner
<code>*wp / wp-></code>	-> CHKA (petq e lock())

MAIN USE: circular reference lucum (parent->child shared, child->parent weak)
observer / cache patterns